

TCASL: Real-Time American Sign Language Classification via Temporal Contrast Emulation

Keshav Shankar

*Electrical and Computer Engineering
University of Pittsburgh
Pittsburgh, United States
keshavshankar@pitt.edu*

Nathaniel Ginck

*Electrical and Computer Engineering
University of Pittsburgh
Pittsburgh, United States
nathanielginck@pitt.edu*

Abstract—American Sign Language (ASL) recognition systems typically rely on standard frame-based cameras, which capture highly redundant pixel data. This redundancy leads to excessive computational overhead, making real-time translation on low-power devices challenging. To address this, we present an end-to-end framework leveraging temporal contrast emulation to transform standard RGB video into sparse, event-based data streams, simulating the behavior of neuromorphic dynamic vision sensors. This approach effectively eliminates static background noise and isolates hand motion. In this paper, we introduce TCASL, a novel event-emulated ASL dataset, alongside a custom Spiking Deep Neural Network (SDNN) optimized for processing sparse temporal data. We evaluate our architecture against state-of-the-art spiking models across multiple event-based datasets. Our SDNN achieves 95.9% accuracy on the TCASL dataset and over 98.1% on standard neuromorphic benchmarks. While the simulated Sigma-Delta neurons incur a slight computational overhead on standard CPUs compared to simpler spiking models, our pipeline maintains a real-time inference speed exceeding 200 frames per second. Finally, we demonstrate the practical viability of this approach through “TCASL Learner,” a gamified, real-time edge-computing application that performs finger-spelling recognition using only a standard consumer webcam.

Index Terms—Spiking Neural Networks, Temporal Contrast Emulation, Dynamic Vision Sensors, Gesture Recognition, American Sign Language

I. INTRODUCTION

American Sign Language (ASL) is the primary language for millions of individuals, yet bridge technologies that translate signs into text often require expensive, high-power hardware. Most current solutions use standard “frame-based” cameras, which record a series of full-color snapshots. This creates a significant problem: the computer must process every pixel in every frame, even if the background (like a wall or a chair) never moves. This leads to high data redundancy, increased heat, and slow performance on mobile devices.

To solve this, we look toward neuromorphic engineering, a field of study inspired by how the human eye and brain process information. Unlike a standard camera, biological vision is asynchronous; our eyes do not wait for a “frame” to finish loading. Instead, our neurons fire only when they detect a change in light intensity. This concept, known as temporal contrast, allows a system to ignore static backgrounds entirely and focus exclusively on the movement of a hand gesture.

While specialized “event cameras” are currently too expensive for widespread use, existing algorithms allow us to emulate this biological process using a standard, low-cost webcam. In this paper, we leverage temporal contrast emulation to transform standard video into a stream of sparse events. Building upon this established technique, we focus on its application to ASL through the development of a specialized dataset and a dedicated spiking neural network designed for real-time classification.

The main contributions of this work are as follows:

- **Dataset:** A novel ASL gesture dataset specifically generated through temporal contrast emulation.
- **SDNN Architecture:** The design of a Spiking Deep Neural Network (SDNN) optimized to classify the unique sparse data produced by our emulation pipeline.
- **Educational Tool:** The implementation of a real-time “Spelling Bee” game application that demonstrates the practical utility of our pipeline.

II. RELATED WORK

A. ASL Recognition

The majority of current ASL recognition systems rely on standard RGB or RGB-D (depth) cameras to capture hand shapes. Recent state-of-the-art methods often employ a combination of Convolutional Neural Networks (CNNs) and Vision Transformers to achieve high accuracy [1], [2]. While effective, these systems are “frame-based,” meaning they capture and process full images at fixed intervals (e.g., 30 frames per second).

This approach has two major drawbacks for real-time use. First, frame-based cameras are highly sensitive to lighting changes and background clutter, which can confuse the model. Second, these models require significant computational power, often necessitating high-end GPUs to process the massive amount of pixel data in each frame [3], [4]. For a simple task like spelling out a name, this level of power consumption is inefficient, especially for mobile or battery-operated devices.

B. Neuromorphic Vision and Event-Based Sensing

To overcome the inefficiencies of traditional cameras, researchers have turned to neuromorphic vision. Unlike standard cameras that record every pixel in a scene regardless of

movement, neuromorphic sensors, often called Event Cameras or Dynamic Vision Sensors (DVS), only record "events" [5]. An event is triggered only when a specific pixel detects a change in brightness, a concept known as temporal contrast [6]. If a hand moves in front of a static wall, the sensor ignores the wall and only transmits data about the moving hand.

While these sensors offer microsecond-level speed and very low power usage, the hardware is currently expensive and difficult for the general public to access. To solve this, researchers use software (e.g., v2e) to emulate event data from standard video frames [7]. This emulation allows us to use the math of neuromorphic vision, focusing only on movement, while still using common hardware like a computer webcam [8].

C. Architectures for Event-Based Classification

Once movement is captured as a stream of events, it must be processed by a neural network. A common approach is to group events into a "frame" and use a standard CNN [9], but this loses the precise timing information that makes event data valuable.

Other researchers represent events as a 3D "cloud" of points [10] or as a connected graph. For instance, Bi et al. [11] introduced a Residual Graph CNN (RG-CNN) that treats ASL events as nodes in a graph. While highly accurate, building these graphs in real-time is mathematically expensive and can slow down the system.

In this work, we utilize Spiking Deep Neural Networks (SDNNs). Unlike traditional networks that stay "on" all the time, SDNNs only process data when they receive a "spike" of activity, much like neurons in the human brain [12]. These networks possess a form of "memory" through their membrane potential, allowing them to naturally understand how a gesture changes over time without the heavy computational cost of graphs or 3D models [13], [14].

III. METHODS

A. Temporal Contrast Emulation

Standard webcams capture full images at fixed intervals, which creates a lot of redundant data. To act more like a human eye, we emulate temporal contrast to detect only movement. As shown in Fig. 1, the system compares the current frame (I_t) to the previous one (I_{t-1}) after converting them to grayscale.

We calculate the difference in brightness for every pixel. If the change is greater than a set threshold (θ), we record it as an "event." If the pixel got brighter, it is an ON event (+1). If it got darker, it is an OFF event (-1). Everything else is ignored (0). The logic is defined as:

$$p(x, y) = \begin{cases} +1 & \text{if } I_t(x, y) - I_{t-1}(x, y) > \theta \\ -1 & \text{if } I_t(x, y) - I_{t-1}(x, y) < -\theta \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This turns real-time video into a sparse map of moving edges, significantly reducing the amount of data the neural network has to process.

B. Dataset Generation

Because no high-quality standard dataset exists for emulated temporal contrast ASL, we built our own: TCASL.

- **Participants:** We recorded data from five different users to ensure the model could recognize different hand shapes and sizes.
- **Classes:** The dataset contains 26 classes (i.e., the letters A–Z). For moving letters (J and Z), we captured the hand's final position to maintain consistency with static gestures.
- **Volume:** Each user provided 100 samples per letter. This resulted in 500 samples per class and a total dataset of 13,000 samples.
- **Format:** All samples were downsized to a resolution of 128×128 pixels to keep the model lightweight.
- **Quality Control:** A manual "expert review" was performed. If a sample was too blurry or the gesture was incorrect, it was deleted to ensure the model only learned from high-quality data.

C. Model Architecture

The core of our system is a Spiking Deep Neural Network (SDNN) designed to process sparse temporal data. As shown in Fig. 3, the network consists of four convolutional layers followed by three fully connected (dense) layers.

Unlike traditional "always-on" neurons, we utilize Sigma-Delta ($\sigma\delta$) neurons. These neurons only communicate when the change in their internal state exceeds a specific threshold (set to 0.1). This ensures that the network remains "silent" unless significant movement is detected.

To improve stability during training, each convolutional layer uses *Mean-Only Batch Normalization*. The final classification is handled by an output layer with 26 neurons, corresponding to the ASL alphabet. To prevent the model from

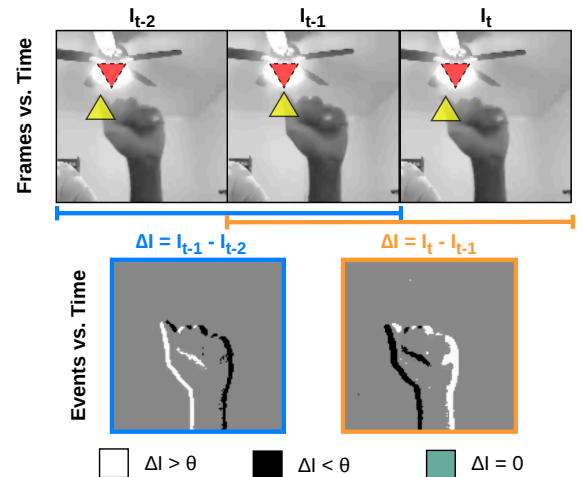


Fig. 1. The process of turning video frames into events.

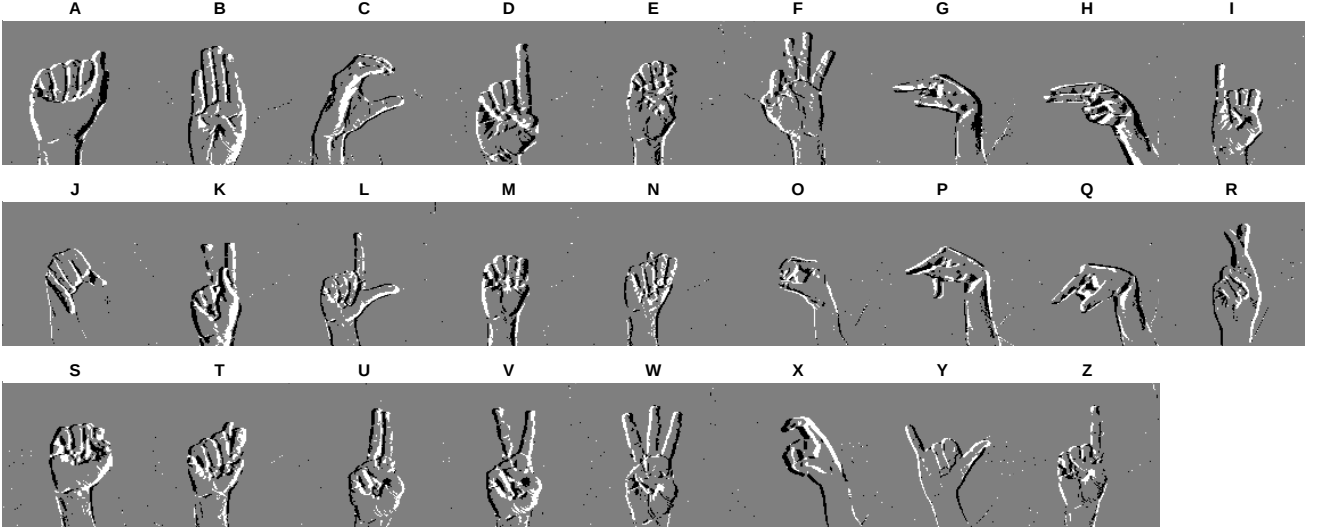


Fig. 2. Samples from the TCASL dataset showing the sparse, edge-based representation of different letters.

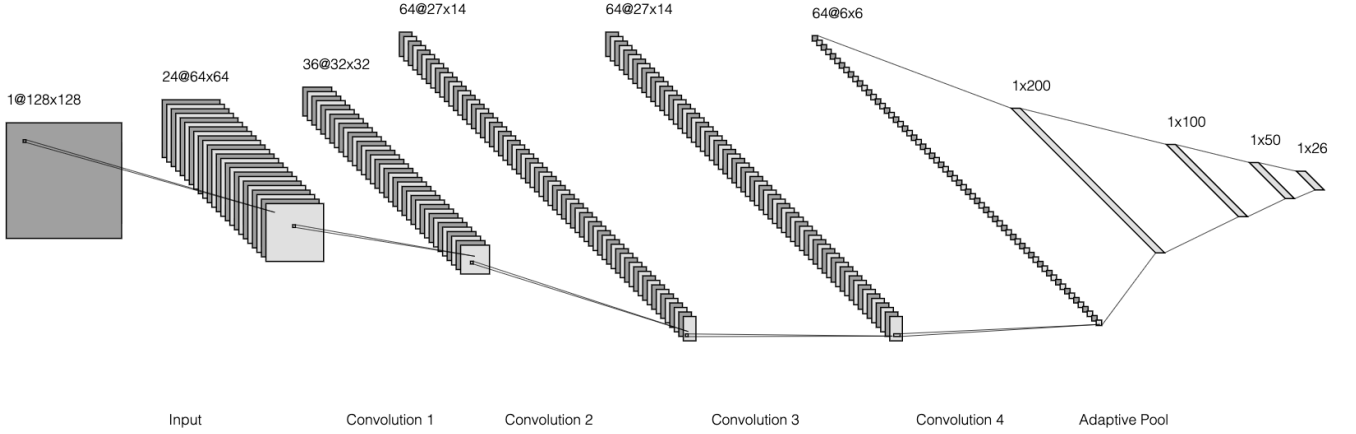


Fig. 3. The SDNN architecture. Input data is delta-encoded, passed through four convolutional stages (24, 36, 64, and 64 filters), and classified via three dense layers.

simply memorizing the training data, we apply a 20% dropout rate to the dense layers.

D. Training Paradigm

Training a spiking network is challenging because "spikes" are discrete events (1 or 0), which makes standard calculus-based learning difficult. We overcome this using two specific techniques:

- 1) **Surrogate Gradient Descent:** During the "forward" pass, the neurons fire discrete spikes. During the "backward" learning pass, we use a smooth relaxation function ($\tau_{grad} = 0.5$) that approximates the spike. This allows us to use the Rectified Adam (RAdam) optimizer to update the network weights.
- 2) **Sparsity-Driven Loss:** Our goal is not just accuracy, but also efficiency. We use a custom loss function, L_{total} ,

which combines standard classification error with an "Event Rate Loss":

$$L_{total} = \text{CrossEntropy} + \lambda \cdot L_{event} \quad (2)$$

The term L_{event} penalizes the network if the neurons fire too frequently. This forces the model to find the "quietest" possible way to recognize a gesture, preserving the power-saving benefits of neuromorphic theory.

We trained the model for 15 epochs with a learning rate of 0.0001 and a batch size of 8, ensuring the weights were optimized for the sparse nature of the TCASL dataset.

IV. SYSTEM IMPLEMENTATION

To demonstrate real-time viability, we developed TCASL Learner, a gamified educational application. The system challenges users to perform ASL finger-spelling in front of a

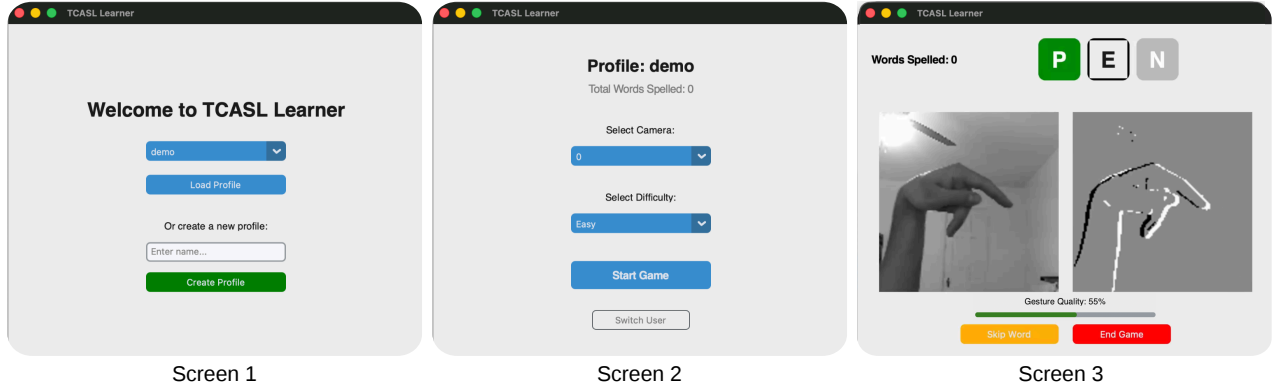


Fig. 4. TCASL Learner interface: (Screen 1) Profile management, (Screen 2) Configuration, and (Screen 3) Real-time gameplay validating the word “PEN”.

standard webcam to complete words in a “Spelling Bee” format.

As illustrated in Fig. 4, the application prompts the user with a target word. The game logic supports three difficulty levels based on word length. The real-time inference pipeline operates as follows:

- 1) **Capture:** Standard 30 FPS webcam input is acquired.
- 2) **Emulation:** Temporal contrast isolates motion into sparse events.
- 3) **Classification:** The SDNN processes these events to predict the letter.
- 4) **Validation:** A sliding window majority-vote mechanism confirms the gesture before progressing to the next character. Average confidence is displayed as “Gesture Quality” to inform the user how well they are signing.

By executing entirely on a standard laptop CPU, this implementation proves that our dataset and pipeline are viable for consumer-grade hardware without requiring specialized neuromorphic sensors.

V. EXPERIMENTAL VALIDATION

A. Dataset Preprocessing

Table I details the statistical differences between the standard benchmark datasets and our custom TCASL dataset. Notably, our pipeline requires significantly fewer total samples to achieve comparable downstream accuracy, highlighting the data efficiency of our temporal contrast emulation.

To ensure a fair and direct comparison across all architectures, the raw event streams from the native NVS datasets (N-MNIST, MNIST-DVS, and ASL-DVS) were standardized to match the temporal contrast emulation format of our custom TCASL dataset. Raw asynchronous event files were accumulated into 2D spatial frames. To mirror standard webcam capabilities, continuous event streams were partitioned into 33-millisecond observation windows to generate 30 frames per second (FPS) sequences.

For all datasets, we employed a uniform temporal contrast visualization scheme: the absence of events was set to a neutral

gray background, while positive (ON) and negative (OFF) polarity spikes were mapped to white and black, respectively. Furthermore, rather than destructively resizing the images to a fixed dimension, we preserved the native spatial resolutions of each dataset by implementing an adaptive global pooling mechanism at the classification head of each model.

B. Experimental Setup

All models were implemented using the *SLAYER* neuromorphic framework within PyTorch. Training was done on an Nvidia H100 GPU. To evaluate real-world inference performance on standard consumer hardware, testing was conducted on an M1 MacBook Pro with 16 GB of RAM. For all architectures, hyperparameters were set to the specific values reported in their respective original literature to ensure a faithful benchmark. The only modification made across all models was the inclusion of the aforementioned adaptive pooling layer.

C. Classification Accuracy

To evaluate the robustness of our architecture and the quality of the TCASL dataset, we perform a dual-benchmark comparison against state-of-the-art architectures.

As shown in Table II, all models performed exceptionally well on the TCASL dataset (> 82%), validating that our temporal contrast emulation provides highly learnable features. Our SDNN achieves strong absolute performance (95.9% on TCASL and 98.1% on N-MNIST/ASL-DVS). However, it is marginally outperformed by the STBP-SNN, which achieved the highest accuracy on three out of the four datasets. This suggests that the strict temporal thresholding of Sigma-Delta neurons may discard subtle spatial features that the Leaky Integrate-and-Fire (LIF) neurons in the STBP-SNN successfully retain.

D. Computational Efficiency and Latency

A core goal of neuromorphic computing is reducing computational overhead. We evaluated performance based on the

TABLE I
DATASET STATISTICS COMPARISON

Metric	N-MNIST	MNIST-DVS	ASL-DVS	TCASL (Ours)
Classes	10	10	24	26
Resolution	34 x 34	128 x 128	240 x 180	128 x 128
Samples / Class	7,000	3,000	4,200	500
Total Samples	70,000	30,000	100,800	13,000

TABLE II
TOP-1 CROSS-MODEL-DATASET ACCURACY COMPARISON

Arch.	N-MNIST [8]	MNIST-DVS [15]	ASL-DVS [11]	TCASL (Ours)
LeNet-5 [16]	0.978	0.733	0.977	0.823
RG-CNN [11]	0.980	0.895	0.981	0.968
STBP-SNN [17]	0.981	0.879	0.984	0.986
Hybrid [2]	0.979	0.862	0.978	0.925
SDNN (Ours)	0.981	0.863	0.981	0.959

total number of trainable parameters and the mean inference latency measured in milliseconds per image.

Contrary to our initial hypotheses regarding the efficiency of SDNNs, Tables III and IV reveal that our SDNN architecture possesses the largest memory footprint (1.17 MB) and the highest inference latency (4.68 ms) among the spiking models tested.

The high parameter count is directly attributable to the three dense classification layers at the tail of the network. Furthermore, while Sigma-Delta neurons are highly energy-efficient when deployed on specialized neuromorphic hardware chips, simulating their complex internal state dynamics via software on a standard von Neumann CPU incurs significant processing overhead. In contrast, the STBP-SNN relies on a leaner convolutional structure and simpler LIF dynamics, resulting in a model that is over three times faster on a CPU. Despite this overhead, the SDNN still achieves over 200 FPS, confirming it is well within the requirements for real-time video processing.

VI. DISCUSSION

A. Interpretation

The results validate that temporal contrast emulation is a highly effective bridge technology. By turning dense video frames into sparse edge-maps, models can achieve near-perfect classification accuracies (up to 98.6% on TCASL) without requiring expensive dedicated event sensors.

However, our benchmarking reveals a critical reality about deploying neuromorphic algorithms on standard hardware: theoretical sparsity does not immediately equal practical speed. While the SDNN was designed to minimize "spike" communication, the heavy use of dense layers and the software overhead of simulating Sigma-Delta math made it slower and larger than simpler spiking models like the STBP-SNN. For applications strictly running on consumer CPUs or mobile phones, leaner network architectures paired with simpler neuron models currently provide a superior balance of accuracy and latency.

B. Current Limitations

The current pipeline faces several practical limitations. First, temporal contrast relies strictly on pixel-level differences. This

makes the system highly sensitive to inconsistent lighting; flickering lights or moving shadows generate false events that can confuse the classifier.

Second, the emulation approach struggles with dynamic gestures. ASL letters like 'J' and 'Z' require continuous motion, but our current system only classifies fixed frames, forcing us to capture only the final position of these signs. Finally, the speed of the user's hand directly impacts the quality of the data. Because events are only generated by movement, a user signing too slowly produces an edge-map that is too sparse for the network to recognize, while signing too quickly causes blurring.

C. Future Work

Future research must address the architectural bloat of the SDNN. We plan to replace the heavy dense layers with Global Average Pooling (GAP) and expand the convolutional backbone, which should drastically reduce the parameter count and improve inference speeds.

Additionally, we intend to transition from static letter classification to dynamic sequence modeling (e.g., using Spiking Recurrent Neural Networks) to properly capture full ASL words and moving letters like 'J' and 'Z'. Finally, to realize the true power-saving benefits of the Sigma-Delta architecture, future iterations of this system will be deployed on physical neuromorphic hardware accelerators (such as Intel Loihi) rather than simulated on traditional CPUs.

VII. CONCLUSION

In this work, we demonstrated that temporal contrast emulation is a highly viable software-level bridge for bringing the efficiency of neuromorphic vision to standard consumer hardware. By filtering out redundant spatial data and focusing purely on asynchronous motion events, we successfully translated the theoretical benefits of dynamic vision sensors into a practical American Sign Language recognition pipeline. We introduced the TCASL dataset and a custom Spiking Deep Neural Network that, despite the software-simulation overhead of Sigma-Delta neurons, achieved up to 95.9%

TABLE III
HARDWARE STORAGE METRICS: PARAMETERS AND MEMORY FOOTPRINT

Arch.	N-MNIST		MNIST-DVS		ASL-DVS		TCASL	
	Parameters	Size (MB)	Parameters	Size (MB)	Parameters	Size (MB)	Parameters	Size (MB)
LeNet-5	61706	0.24	61706	0.24	62896	0.24	63066	0.24
RG-CNN	285706	1.10	285706	1.10	287512	1.11	287770	1.11
STBP-SNN	210740	0.81	210740	0.81	214954	0.82	215556	0.83
Hybrid	161354	0.63	161354	0.63	163160	0.63	163418	0.64
SDNN	301968	1.17	301968	1.17	303376	1.17	303376	1.17

TABLE IV
HARDWARE INFERENCE SPEED: LATENCY AND THROUGHPUT

Arch.	N-MNIST		MNIST-DVS		ASL-DVS		TCASL	
	Latency (ms)	FPS	Latency (ms)	FPS	Latency (ms)	FPS	Latency (ms)	FPS
LeNet-5	0.75	1338	0.59	1668	0.66	1518	0.69	1447
RG-CNN	4.97	201	5.21	191	5.54	180	4.92	203
STBP-SNN	1.47	678	1.36	735	1.59	740	1.35	740
Hybrid	2.87	348	2.29	437	2.89	346	2.78	359
SDNN	4.30	232	4.68	213	4.74	210	4.68	213

classification accuracy and operated at over 200 FPS on a standard CPU. The deployment of this pipeline into the real-time “TCASL Learner” educational application proves that event-based methodologies can run efficiently at the edge without the need for expensive, specialized cameras or high-end GPUs. While challenges remain in handling dynamic sequence modeling and optimizing dense architectural components for CPU execution, this research establishes a strong foundation for accessible, privacy-preserving, and low-latency continuous sign language translation.

REFERENCES

- [1] B. Alsharif, M. Alanazi, A. S. Altaher, A. Altaher, and M. Ilyas, “Deep learning technology to recognize american sign language alphabet using mulit-focus image fusion technique,” in *2023 IEEE 20th International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET)*. IEEE, 2023, pp. 1–6.
- [2] M. Aly and I. S. Fathi, “Recognizing american sign language gestures efficiently and accurately using a hybrid transformer model,” *Scientific Reports*, vol. 15, no. 1, p. 20253, 2025.
- [3] D. Biyani, N. V. Doohan, M. Rode, and D. Jain, “Real time sign language recognition using yolov5,” in *2023 IEEE World Conference on Applied Intelligence and Computing (AIC)*. IEEE, 2023, pp. 582–588.
- [4] A. A. Barbhuiya, R. K. Karsh, and R. Jain, “Cnn based feature extraction and classification for sign language,” *Multimedia Tools and Applications*, vol. 80, no. 2, pp. 3051–3069, 2021.
- [5] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis *et al.*, “Event-based vision: A survey,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 1, pp. 154–180, 2020.
- [6] C. Posch, D. Matolin, and R. Wohlgenannt, “A qvga 143 db dynamic range frame-free pwm image sensor with lossless pixel-level video compression and time-domain cds,” *IEEE Journal of Solid-State Circuits*, vol. 46, no. 1, pp. 259–275, 2010.
- [7] Y. Hu, S.-C. Liu, and T. Delbruck, “v2e: From video frames to realistic dvs events,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 1312–1321.
- [8] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, “Converting static image datasets to spiking neuromorphic datasets using saccades,” *Frontiers in neuroscience*, vol. 9, p. 437, 2015.
- [9] D. Gehrig, A. Loquercio, K. G. Derpanis, and D. Scaramuzza, “End-to-end learning of representations for asynchronous event-based data,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 5633–5643.
- [10] Q. Wang, Y. Zhang, J. Yuan, and Y. Lu, “Space-time event clouds for gesture recognition: From rgb cameras to event cameras,” in *2019 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2019, pp. 1826–1835.
- [11] Y. Bi, A. Chadha, A. Abbas, E. Bourtsoulatz, and Y. Andreopoulos, “Graph-based object classification for neuromorphic vision sensing,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 491–501.
- [12] G. Orchard, C. Meyer, R. Etienne-Cummings, C. Posch, N. Thakor, and R. Benosman, “Hfirst: A temporal approach to object recognition,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 37, no. 10, pp. 2028–2040, 2015.
- [13] X. Lagorce, G. Orchard, F. Galluppi, B. E. Shi, and R. B. Benosman, “Hots: a hierarchy of event-based time-surfaces for pattern recognition,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 39, no. 7, pp. 1346–1359, 2016.
- [14] M. Cannici, M. Ciccone, A. Romanoni, and M. Matteucci, “A differentiable recurrent surface for asynchronous event-based data,” in *European Conference on Computer Vision*. Springer, 2020, pp. 136–152.
- [15] T. Serrano-Gotarredona and B. Linares-Barranco, “Poker-dvs and mnist-dvs. their history, how they were made, and other details,” *Frontiers in neuroscience*, vol. 9, p. 481, 2015.
- [16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.
- [17] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, “Spatio-temporal backpropagation for training high-performance spiking neural networks,” *Frontiers in neuroscience*, vol. 12, p. 331, 2018.